

Architectural Significant Requirement (ASR)

Movie Explorer System

1. Frontend Architecture

The frontend layer of my Movie Explorer system is designed as a modern Single Page Application (SPA) that prioritizes responsiveness, usability, and scalability. The system is implemented using React.js, enabling a component-based architecture where UI elements are modular, reusable, and independently maintainable.

The user interface is divided into multiple functional components including navigation bars, search modules, filtering systems, movie display cards, and personalized dashboards. Each component is designed with separation of concerns, ensuring that UI logic, business logic, and styling remain decoupled.

State management is handled using React's useState hook, ensuring predictability and simplicity. This is particularly important for managing global states such as user authentication, search queries, and watchlists.

Performance optimization techniques such as lazy loading, code splitting, and memoization are implemented to enhance responsiveness. These techniques reduce initial load time and improve runtime efficiency.

Security considerations include input validation, protection against Cross-Site Scripting (XSS), and secure handling of authentication tokens using HTTP-only cookies.

The frontend communicates with backend services through RESTful APIs, ensuring a loosely coupled architecture. Responsive design principles are implemented using Tailwind CSS to ensure compatibility across devices.

2. Backend Architecture

The backend architecture of Movie Explorer is built as a scalable and modular service-oriented system. It is implemented using Node.js with Express.js, providing a lightweight yet powerful environment for handling concurrent requests.

The architecture follows a layered design pattern consisting of controllers, services, and data access layers. This separation enhances maintainability and allows independent scaling of components.

The backend is responsible for user authentication, data processing, business logic execution, and integration with third-party APIs such as TMDB(The Movie Database).

Authentication is implemented using JSON Web Tokens (JWT), ensuring secure and stateless communication between client and server.

To enhance performance, asynchronous programming and non-blocking I/O operations are used extensively. Redis caching is integrated to reduce redundant API calls and improve response times.

Security measures include rate limiting, input sanitization, secure API endpoints, and HTTPS communication. Logging mechanisms are also integrated to monitor system behavior.

3. DevOps Architecture

The DevOps strategy for Movie Explorer emphasizes simplicity, reliability, and efficient development workflows, tailored to the current scale of the application. Version control is managed using Git, with repositories hosted on GitHub, enabling collaborative development, version tracking, and code integrity.

A lightweight CI/CD approach is implemented using GitHub Actions, which automates essential processes such as code integration, basic testing, and deployment. This ensures that updates to the application are consistently validated and can be deployed with minimal manual intervention.

The deployment workflow primarily relies on platform-managed environments (e.g., Vercel for frontend and cloud hosting services for backend), reducing the need for complex infrastructure management. This approach allows rapid development and easy deployment while maintaining system reliability.

Monitoring and debugging are handled through built-in logging mechanisms and platform-provided analytics tools. These tools help track application performance, identify errors, and ensure system stability during runtime.

Basic deployment safeguards, including version control rollback and error monitoring, are used to maintain application stability and ensure quick recovery from failures.

4. Database Architecture

The database architecture of Movie Explorer is designed around a relational database model using PostgreSQL, ensuring data consistency, integrity, and structured storage of application data. PostgreSQL is selected due to its robustness, strong ACID compliance, and suitability for handling structured relationships within the system.

The core data entities include users, movies, watchlists, and ratings. These entities are organized using a normalized schema to minimize redundancy and maintain data consistency. Relationships such as one-to-many (e.g., users to watchlists) and many-to-many (e.g., users and movies through ratings or watchlists) are carefully structured using foreign keys and relational constraints.

To enhance performance, indexing strategies are applied to frequently queried fields such as user IDs and movie IDs. This ensures efficient query execution, particularly for operations involving search, filtering, and retrieval of user-specific data.

Database operations are performed using structured SQL queries, allowing precise control over data manipulation and retrieval. This approach provides transparency and flexibility in managing the database layer without introducing additional abstraction layers.

Data integrity is enforced through constraints such as primary keys, foreign keys, and validation rules defined at the database level. These mechanisms ensure that only valid and consistent data is stored within the system.

Basic backup strategies, including periodic database dumps, can be employed to ensure data recovery and system reliability.

5. Deployment Environment

The deployment environment of Movie Explorer is designed to provide a reliable, accessible, and scalable platform using modern cloud-based services tailored to the current scope of the application.

The frontend application is deployed on **Vercel**, which provides fast global content delivery, automatic builds, and seamless integration with GitHub repositories. This ensures that updates to the user interface are deployed efficiently with minimal configuration, while also benefiting from edge network performance optimization.

The backend services are hosted on **Render**, which supports continuous deployment and simplifies server management. The platform automatically builds and deploys backend updates upon code changes, allowing for streamlined development and deployment workflows without requiring manual infrastructure configuration.

The database layer is managed using **Neon**, a cloud-based PostgreSQL service that provides scalability, automated backups, and high availability. This enables efficient data management while offloading operational complexity associated with traditional database administration.

Communication between the frontend, backend, and database is conducted over secure HTTPS connections, ensuring data integrity and confidentiality during transmission.

The deployment workflow leverages GitHub-based version control, where code changes trigger automated deployment processes on both Vercel and Render. This establishes a lightweight Continuous Deployment (CD) pipeline suitable for rapid iteration and development.